

**UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS**

Caio Wingeter de Castilho

**Otimização de Baixo Nível em Vision Transformers:
Desenvolvimento de Kernels de Inferência Customizados
para Eficiência de SRAM**

São Carlos

2026

Caio Wingeter de Castilho

**Otimização de Baixo Nível em Vision Transformers:
Desenvolvimento de Kernels de Inferência Customizados
para Eficiência de SRAM**

Monografia apresentada ao Curso de Engenharia Aeronáutica, da Escola de Engenharia de São Carlos da Universidade de São Paulo, como parte dos requisitos para obtenção do título de Engenheiro Aeronáutico.

Orientador: Prof. Dr. Glauco Caurin

VERSÃO CORRIGIDA

**São Carlos
2026**

É possível elaborar a ficha catalográfica em LaTeX ou incluir a fornecida pela Biblioteca. Para tanto observe a programação contida nos arquivos USPSC-modelo.tex e fichacatalografica.tex e/ou gere o arquivo fichacatalografica.pdf.

A biblioteca da sua Unidade lhe fornecerá um arquivo PDF com a ficha catalográfica definitiva, que deverá ser salvo como fichacatalografica.pdf no diretório do seu projeto.

ERRATA

A errata é um elemento opcional, que consiste de uma lista de erros da obra, precedidos pelas folhas e linhas onde eles ocorrem e seguidos pelas correções correspondentes. Deve ser inserida logo após a folha de rosto e conter a referência do trabalho para facilitar sua identificação, conforme a ABNT NBR 14724 (??).

Modelo de Errata:

CASTILHO, C. W. **Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM.** 2026. 61 p. Monografia (Trabalho de Conclusão de Curso) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

ERRATA

Folha	Linha	Onde se lê	Leia-se
1	10	auto-conclavo	autoconclavo

Folha de aprovação em conformidade
com o padrão definido
pela Unidade.

No presente modelo consta como
folhadeaprovacao.pdf

*Este trabalho é dedicado à todos que me apoiaram
e me deram o suporte durante esse percurso.*

AGRADECIMENTOS

Dedico este trabalho a todos que estiveram ao meu lado ao longo dessa caminhada, oferecendo apoio, força e incentivo para que eu pudesse superar cada desafio encontrado no percurso.

Agradeço, de forma especial, à minha companheira Luana, por todo o carinho, paciência e apoio incondicional ao longo dessa jornada. Aos meus pais, André e Carolina, que, apesar de todas as dificuldades, nunca deixaram de acreditar em mim e sempre se esforçaram ao máximo para que eu pudesse alcançar meus sonhos. Ao meu irmão Breno, que esteve ao meu lado em todos os momentos, não apenas como irmão, mas como meu melhor amigo.

*“Líderes são mestres em encontrar o sentido e aprendizado
em todo tipo de experiência de vida.”*
(Jucá, 2014, p. 94)

RESUMO

CASTILHO, C. W. **Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM.** 2026. 61 p. Monografia (Trabalho de Conclusão de Curso) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

Este trabalho aborda a otimização de baixo nível de kernels de inferência aplicados a modelos de visão computacional, com foco específico na arquitetura Vision Transformer (ViT). O objetivo principal consiste em reduzir o gargalo de transferência de dados entre a memória de alta largura de banda (HBM) e os processadores de fluxo (SM) da GPU, fenômeno associado à “Memory Wall”. A metodologia fundamenta-se no desenvolvimento de um kernel de atenção fundido utilizando OpenAI Triton, explorando estratégias de tiling, reuso de dados em registradores e SRAM, e cálculo de softmax online para evitar a materialização completa da matriz intermediária de atenção. A validação proposta compara a implementação customizada com baselines em PyTorch, utilizando métricas de latência, throughput, erro numérico, tráfego de memória e análise pelo Modelo Roofline. Espera-se, com isso, avaliar em que medida a fusão de operadores e a consciência da hierarquia de memória podem aproximar a execução da atenção dos limites efetivos do hardware. O trabalho contribui com uma implementação experimental e um protocolo de análise para investigar gargalos de desempenho em kernels de atenção aplicados a Vision Transformers.

Palavras-chave: Visão Computacional. Vision Transformers. Otimização de Kernels. OpenAI Triton. Engenharia de Performance.

ABSTRACT

CASTILHO, C. W. **Hardware-Aware Optimization of Vision Transformers: Implementing High-Performance Inference Kernels for SRAM Efficiency**. 2026. 61 p. Monograph (Conclusion Course Paper) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

This work addresses the low-level optimization of inference kernels applied to computer vision models, with a specific focus on the Vision Transformer (ViT) architecture. The primary objective is to reduce the data transfer bottleneck between High Bandwidth Memory (HBM) and the GPU's Streaming Multiprocessors (SM), a phenomenon associated with the "Memory Wall." The methodology is based on the development of a fused attention kernel using OpenAI Triton, exploring tiling strategies, data reuse in registers and SRAM, and online softmax computation to avoid fully materializing the intermediate attention matrix. The proposed validation compares the custom implementation against PyTorch baselines using latency, throughput, numerical error, memory traffic, and Roofline analysis. The goal is to evaluate to what extent operator fusion and awareness of the memory hierarchy can bring attention execution closer to the effective hardware limits. This work contributes an experimental implementation and an analysis protocol for investigating performance bottlenecks in attention kernels applied to Vision Transformers.

Keywords: Computer Vision. Vision Transformers. Kernel Optimization. OpenAI Triton. Performance Engineering.

LISTA DE FIGURAS

Figura 1 – Arquitetura do Vision Transformer (ViT)	33
Figura 2 – Mecanismos de atenção do Transformer	34
Figura 3 – Hierarquia de memória com largura de banda e capacidade	36
Figura 4 – Esquema do FlashAttention e comparação de tempo de execução . . .	39
Figura 5 – Acentuação (modo texto - L ^A T _E X)	61

LISTA DE TABELAS

LISTA DE QUADROS

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
EESC	Escola de Engenharia de São Carlos
GPU	Graphics Processing Unit
HBM	High Bandwidth Memory
HPC	High Performance Computing
ML	Machine Learning
PDF	Portable Document Format
SM	Streaming Multiprocessors
SRAM	Static Random Access Memory
TCC	Trabalho de Conclusão de Curso
USP	Universidade de São Paulo
ViT	Vision Transformer

LISTA DE SÍMBOLOS

Q	Matriz <i>Query</i>
K	Matriz <i>Key</i>
V	Matriz <i>Value</i>
K^T	Transposta da matriz K
d_k	Dimensionalidade dos vetores de chave
N	Número de <i>patches</i> (tokens) da entrada
$O(N^2)$	Complexidade computacional quadrática em função de N

SUMÁRIO

1	INTRODUÇÃO	29
1.1	Contexto e delimitação do tema	29
1.2	Justificativa	29
1.3	Problema de pesquisa	30
1.4	Hipótese de trabalho e contribuições esperadas	30
1.5	Objetivos	30
1.6	Delimitação operacional	31
1.7	Organização do trabalho	31
2	DESENVOLVIMENTO	33
2.1	Vision Transformers e a Mecânica de Atenção	33
2.1.1	Por que Autoatenção?	34
2.2	Complexidade Computacional e Movimentação de Dados na Atenção	35
2.3	Arquitetura de GPU e a “Parede de Memória” (Memory Wall)	36
2.3.1	Modelo de execução em GPUs	37
2.4	Modelo Roofline para Análise de Desempenho	37
2.5	OpenAI Triton	38
2.6	Fusão de Operadores e Flash Attention	38
3	METODOLOGIA	41
3.1	Ambiente Experimental	41
3.2	Design do Kernel Customizado	41
3.3	Implementação do Softmax Online	42
3.4	Protocolo de Avaliação	43
4	RESULTADOS E DISCUSSÕES	45
4.1	Análise Comparativa de Latência	45
4.2	Profiling de Baixo Nível	45
4.3	Modelo Roofline	45
5	CONCLUSÃO	47
	REFERÊNCIAS	49

APÊNDICES **51**

**APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E
SOFTMAX COM TRITON 53**

A.1 Implementação padrão em PyTorch 53

A.2 Implementação otimizada com Triton 54

A.3 Comparação técnica 56

ANEXOS **57**

ANEXO A – EXEMPLO DE ANEXO 59

ANEXO B – ACENTUAÇÃO (MODO TEXTO - $\text{\LaTeX}$) 61

1 INTRODUÇÃO

O avanço exponencial dos modelos de Aprendizado de Máquina (ML), particularmente na área de Visão Computacional, tem sido impulsionado pela transição de arquiteturas convolucionais para os Vision Transformers (ViTs) (Dosovitskiy *et al.*, 2021). Diferente das convoluções, que operam em domínios locais, os ViTs utilizam o mecanismo de autoatenção (Self-Attention) para capturar dependências globais em uma imagem (Vaswani *et al.*, 2023). No entanto, essa capacidade analítica superior impõe um custo computacional elevado e uma demanda intensiva de memória, apresentando desafios significativos para a implantação desses modelos em ambientes de produção de alta performance (Dao *et al.*, 2022).

1.1 Contexto e delimitação do tema

Atualmente, o gargalo da computação de alto desempenho (HPC) em Inteligência Artificial não reside apenas na capacidade aritmética das Unidades de Processamento Gráfico (GPUs), mas na eficiência da movimentação de dados. Este fenômeno, conhecido como Memory Wall, ocorre quando a velocidade de transferência entre a Memória de Alta Largura de Banda (HBM) e os Processadores de Fluxo (SM) da GPU é insuficiente para manter os núcleos de processamento (Tensor Cores) ocupados (Wulf; McKee, 1996).

No desenvolvimento de kernels de inferência, a maioria dos frameworks de alto nível, como PyTorch e TensorFlow, utiliza implementações genéricas. Embora versáteis, essas implementações frequentemente realizam múltiplos acessos à memória global para operações sequenciais. Esta pesquisa delimita-se à otimização de baixo nível desses processos, focando na camada de atenção de Vision Transformers, utilizando a linguagem de programação de kernels OpenAI Triton para maximizar o aproveitamento da hierarquia de memória do hardware (Tillet; Kung; Cox, 2019).

1.2 Justificativa

A atenção densa é uma das operações centrais dos Transformers e, no caso dos Vision Transformers, seu custo cresce quadraticamente com o número de patches da imagem. Esse comportamento torna-se particularmente relevante em cenários de inferência, nos quais a latência por amostra, o consumo de memória e o custo de ocupação da GPU influenciam diretamente a viabilidade de implantação do modelo. Mesmo quando a quantidade de operações aritméticas é compatível com a capacidade nominal do hardware, a execução pode permanecer limitada pelo tráfego de dados entre HBM e os níveis internos de memória da GPU.

Nesse contexto, a otimização de kernels deixa de ser apenas um detalhe de implementação e passa a ser parte do próprio projeto de sistemas de aprendizado profundo. Um kernel fundido para atenção permite reduzir a materialização de tensores intermediários, diminuir acessos à memória global e explorar melhor o paralelismo disponível nos SMs. A relevância do trabalho, portanto, está em conectar a formulação algorítmica dos ViTs com decisões concretas de engenharia de performance, aproximando a análise teórica da operação de atenção do comportamento observado no hardware.

1.3 Problema de pesquisa

A implementação padrão da operação de atenção em ViTs possui uma complexidade quadrática em relação ao número de patches da imagem (Vaswani *et al.*, 2023; Dosovitskiy *et al.*, 2021). Durante a inferência, a necessidade de armazenar e ler matrizes intermediárias de atenção da memória global (VRAM) gera uma latência considerável e um consumo energético ineficiente (Dao *et al.*, 2022). Surge, portanto, a questão: como o desenvolvimento de kernels customizados e a aplicação de técnicas de fusão de operadores podem reduzir a dependência da memória global e aproximar a performance dos modelos de visão do limite teórico do hardware?

1.4 Hipótese de trabalho e contribuições esperadas

A hipótese deste trabalho é que uma implementação de atenção consciente da hierarquia de memória, baseada em tiling e fusão de operadores, reduz o tráfego de memória global em relação a uma implementação ingênua que materializa explicitamente a matriz de atenção. Espera-se que essa redução de tráfego se traduza em menor latência de inferência e maior throughput efetivo, sobretudo para sequências maiores, nas quais o custo quadrático da atenção amplifica o impacto da matriz intermediária $N \times N$.

Como contribuição prática, o trabalho propõe a implementação de um kernel de atenção em OpenAI Triton, acompanhado de um protocolo experimental para comparação com baselines em PyTorch. Como contribuição analítica, propõe-se interpretar os resultados por meio de métricas de latência, throughput, tráfego de memória e Modelo Roofline, distinguindo se o gargalo observado é dominado por largura de banda de memória ou capacidade computacional.

1.5 Objetivos

O objetivo central deste trabalho é desenvolver, implementar e validar um kernel de inferência de alto desempenho para a operação de Self-Attention em Vision Transformers.

Sendo seus objetivos específicos:

- Desenvolver um kernel fundido (fused kernel) utilizando OpenAI Triton que integre as operações de produto escalar, softmax e ponderação de valores em uma execução com reuso de dados em registradores e SRAM;
- Implementar estratégias de tiling para otimizar o carregamento de blocos de dados nos registradores da GPU;
- Realizar um profiling comparativo entre a implementação proposta e o baseline do PyTorch, utilizando métricas de latência, throughput (TFLOP/s) e utilização de banda de memória;
- Analisar a eficiência do kernel desenvolvido através do Modelo Roofline para identificar gargalos operacionais remanescentes.

1.6 Delimitação operacional

O escopo experimental limita-se ao forward pass da atenção densa não causal, sem dropout e voltada à inferência. As entradas são consideradas no formato usual de atenção multi-cabeças, com tensores de Query, Key e Value já projetados pela camada linear anterior. Não fazem parte do escopo inicial o treinamento completo de um ViT, o cálculo de gradientes, mecanismos de atenção esparsa, quantização agressiva ou otimizações específicas de pré-processamento de imagens.

Essa delimitação permite concentrar a análise no núcleo computacional de maior interesse para o trabalho: a sequência de operações QK^T , softmax e multiplicação por V . Dessa forma, os resultados podem ser interpretados com menor interferência de camadas adjacentes do modelo e de custos externos à operação de atenção.

1.7 Organização do trabalho

O Capítulo 1 apresenta o contexto, o problema de pesquisa, a hipótese de trabalho e os objetivos. O Capítulo 2 discute os fundamentos técnicos necessários para a proposta, incluindo Vision Transformers, complexidade da atenção, arquitetura de GPU, Triton, FlashAttention e Modelo Roofline. Em seguida, a Metodologia descreve o ambiente experimental, o desenho do kernel customizado e o protocolo de avaliação. O capítulo de Resultados e Discussões é reservado à análise dos benchmarks e do profiling de baixo nível. Por fim, a Conclusão sintetiza os achados e aponta limitações e possibilidades de continuidade.

2 DESENVOLVIMENTO

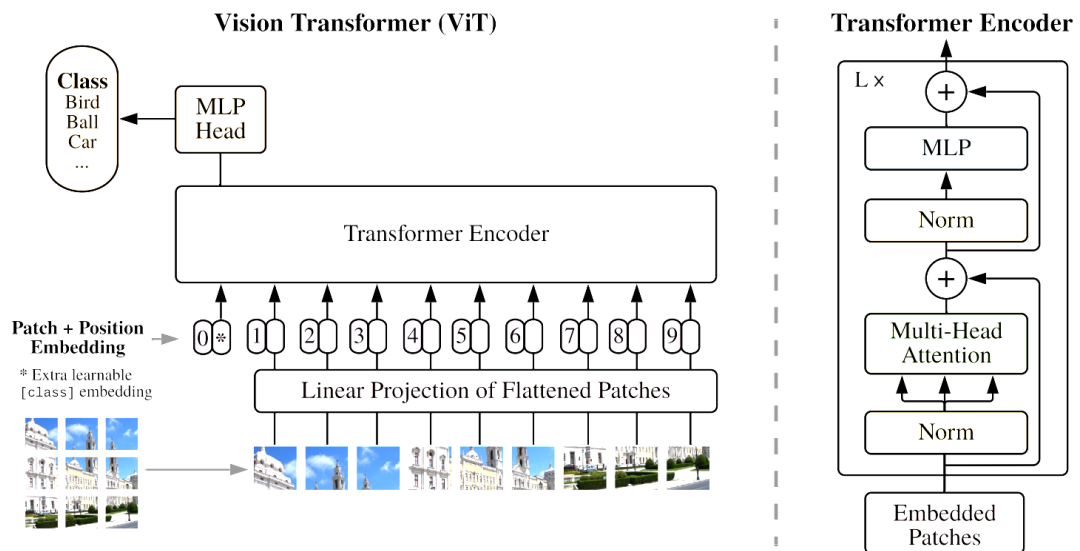
Neste capítulo, apresentam-se os conceitos fundamentais para a compreensão da otimização de kernels de inferência. A discussão abrange desde a arquitetura dos Vision Transformers até as minúcias da hierarquia de memória em Unidades de Processamento Gráfico (GPUs), culminando nas ferramentas de programação de baixo nível.

2.1 Vision Transformers e a Mecânica de Atenção

Os Vision Transformers (ViT) representam uma mudança de paradigma na visão computacional ao transpor a arquitetura de autoatenção, originalmente concebida para processamento de linguagem natural (Vaswani *et al.*, 2023), para o domínio de imagens. Segundo Dosovitskiy *et al.* (2021), o ViT decompõe uma imagem em patches fixos, os quais são tratados como sequências de tokens, permitindo que o modelo aprenda relações globais entre diferentes regiões da cena desde as camadas iniciais.

Como referência visual da arquitetura, a Figura 1 apresenta o fluxo do ViT, desde a tokenização em patches até o codificador Transformer e a cabeça de classificação.

Figura 1 – Arquitetura do Vision Transformer (ViT)



Fonte: Adaptado de Dosovitskiy *et al.* (2021).

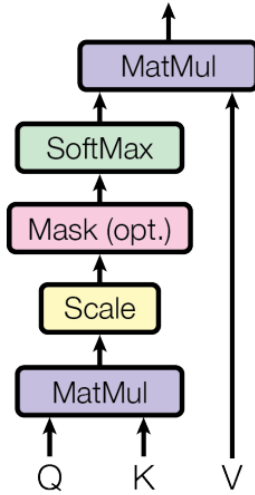
O cerne dessa arquitetura é o mecanismo de autoatenção, cujos componentes fundamentais, o Scaled Dot-Product Attention e sua extensão para o Multi-Head Attention,

são ilustrados na Figura 2. Matematicamente, a operação de atenção é definida pela Equação 2.1:

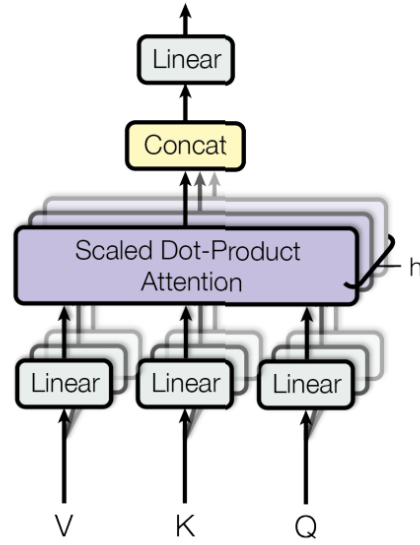
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.1)$$

Figura 2 – Mecanismos de atenção do Transformer

(a) Scaled Dot-Product Attention



(b) Multi-Head Attention



Fonte: Vaswani *et al.* (2023).

Onde Q , K e V representam as matrizes de Query, Key e Value. Embora eficaz, essa operação apresenta uma complexidade computacional $O(N^2)$, onde N é o número de patches. Como apontam Dao *et al.* (2022), o gargalo dessa operação em GPUs modernas não é o poder computacional bruto, mas sim a latência de memória gerada pela leitura e escrita de matrizes intermediárias de atenção.

2.1.1 Por que Autoatenção?

No trabalho original dos Transformers, Vaswani *et al.* (2023) justificam o uso da autoatenção comparando-a com camadas recorrentes e convolucionais para o mapeamento de uma sequência de entrada em outra de mesmo comprimento. A análise é guiada por três critérios centrais: (i) complexidade computacional por camada, (ii) grau de paralelização e (iii) comprimento do caminho entre dependências de longo alcance.

Sob essa ótica, a autoatenção apresenta uma vantagem estrutural importante: todas as posições da sequência interagem diretamente em uma única camada, com número constante de operações sequenciais. Em contraste, arquiteturas recorrentes exigem dependência

temporal passo a passo, com custo sequencial proporcional ao comprimento da sequência. Na prática, isso reduz o paralelismo possível em GPU e limita o throughput.

Também no aspecto de dependências de longo alcance, a autoatenção encurta o caminho de propagação de informação entre dois tokens para ordem constante. Em arquiteturas recorrentes, esse caminho cresce linearmente com o tamanho da sequência; em convolucionais, depende da largura do kernel e da profundidade da pilha de camadas. Essa diferença é relevante porque caminhos menores tendem a facilitar o aprendizado de relações globais.

Em termos de custo assintótico, a autoatenção densa envolve tipicamente $O(n^2d)$, enquanto camadas recorrentes escalam como $O(nd^2)$ e convoluções dependem de k (largura do kernel). Conforme discutido por Vaswani *et al.* (2023), quando $n < d$, cenário comum em representações de linguagem e compatível com várias configurações de tokens em visão, a autoatenção tende a ser computacionalmente mais favorável que abordagens recorrentes, além de ser mais paralelizável.

Por fim, os autores destacam que versões restritas de autoatenção (janela local de tamanho r) podem reduzir custo para sequências muito longas, ao custo de aumentar o caminho máximo para $O(n/r)$. Esse compromisso custo–alcance é particularmente útil para conectar a formulação teórica da atenção com o problema de engenharia tratado neste trabalho: minimizar movimentação de dados e latência de memória sem perder capacidade de modelar dependências globais.

2.2 Complexidade Computacional e Movimentação de Dados na Atenção

A Equação 2.1 evidencia que a atenção densa é composta por três etapas principais: o produto QK^T , a normalização por *softmax* e a multiplicação da matriz de probabilidades por V . Considerando uma cabeça de atenção com N tokens e dimensão D , o produto QK^T produz uma matriz de scores de dimensão $N \times N$ e possui custo aproximado de $2N^2D$ operações de ponto flutuante. A multiplicação final por V possui custo da mesma ordem, também próximo de $2N^2D$. Assim, desconsiderando constantes associadas ao *softmax*, o custo aritmético do forward de uma cabeça é da ordem de $O(N^2D)$.

O custo aritmético, entretanto, não descreve completamente o desempenho em GPU. Em uma implementação ingênua, a matriz QK^T é escrita na memória global, posteriormente lida para a aplicação do *softmax*, novamente escrita como matriz de probabilidades e, por fim, lida para a multiplicação por V . Esse fluxo materializa um tensor intermediário de tamanho $N \times N$ que pode dominar o tráfego em HBM quando N cresce. Em ViTs, esse efeito aparece de forma direta: dobrar a resolução espacial ou reduzir o tamanho dos patches aumenta o número de tokens e amplia quadraticamente a matriz de atenção.

Para uma configuração com B amostras, H cabeças, N tokens e dimensão de cabeça D , o custo aritmético aproximado do forward de atenção densa pode ser representado por:

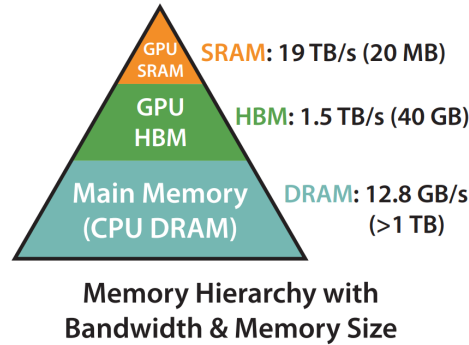
$$FLOPs \approx 4BHN^2D. \quad (2.2)$$

Já a matriz intermediária de scores ocupa, sozinha, BHN^2 elementos. Em FP16, isso corresponde a aproximadamente $2BHN^2$ bytes por materialização. Como uma implementação não fundida pode escrever e reler essa estrutura em mais de uma etapa, o tráfego efetivo de memória pode superar o tamanho dos tensores de entrada e saída. Essa diferença explica por que otimizações como FlashAttention e kernels fundidos não buscam apenas reduzir FLOPs, mas reduzir operações de entrada e saída na memória global (Dao *et al.*, 2022).

2.3 Arquitetura de GPU e a “Parede de Memória” (Memory Wall)

Para entender a necessidade de otimização de baixo nível, é preciso distinguir os diferentes níveis de memória em uma GPU. A memória principal, conhecida como High Bandwidth Memory (HBM), possui alta capacidade, porém latência significativamente superior à memória estática interna do chip, a SRAM.

Figura 3 – Hierarquia de memória com largura de banda e capacidade



Fonte: Adaptado de Dao *et al.* (2022).

O desequilíbrio entre a velocidade de processamento e a velocidade de transferência de dados é o que a literatura denomina como Memory Wall. Sobre este desafio, Wulf e McKee (1996) observam:

[...] a disparidade entre a velocidade dos processadores e a velocidade da memória principal tem crescido exponencialmente. Se essa tendência continuar, a performance dos sistemas será limitada não pela frequência de clock do processador, mas pela taxa na qual os dados podem ser entregues a ele (Wulf; McKee, 1996) (tradução do autor).

Dessa forma, otimizar um kernel significa maximizar o tempo que o dado permanece na SRAM (ou Shared Memory), minimizando acessos à HBM. Em implementações padrão de bibliotecas de alto nível, cada etapa da Equação 2.1 resulta em uma escrita e leitura na HBM, o que degrada o throughput do sistema.

2.3.1 Modelo de execução em GPUs

Uma GPU executa milhares de threads organizadas em blocos, que são escalonados nos Streaming Multiprocessors. Cada SM possui recursos limitados de registradores, memória compartilhada, unidades de execução e slots para warps residentes. O desempenho de um kernel depende, portanto, de uma combinação entre paralelismo, ocupação, coalescência de acessos à memória e reutilização local de dados.

Threads são agrupadas em warps, que executam a mesma instrução sobre dados distintos. Quando os acessos à memória global são contíguos e alinhados, a GPU consegue agrupá-los em transações mais eficientes. Por outro lado, acessos irregulares, baixa reutilização de dados ou excesso de registradores por thread podem reduzir a ocupação e aumentar a exposição à latência de memória. Em kernels de atenção, esses fatores aparecem na escolha do tamanho dos blocos de Q , K e V , no número de warps por programa e na quantidade de dados mantida em registradores ou memória compartilhada.

Para o presente trabalho, essa visão é importante porque o objetivo não é apenas reescrever uma equação matemática em outra linguagem. O objetivo é organizar a mesma computação de modo que cada bloco carregado da HBM seja reutilizado tantas vezes quanto possível antes de ser descartado. A eficiência do kernel depende, assim, da relação entre trabalho aritmético realizado e bytes transferidos pela hierarquia de memória.

2.4 Modelo Roofline para Análise de Desempenho

O Modelo Roofline é uma ferramenta de análise que relaciona o desempenho atingível de uma aplicação com sua intensidade aritmética, isto é, a razão entre operações de ponto flutuante e bytes movimentados na memória (Williams; Waterman; Patterson, 2009). Nesse modelo, o desempenho máximo de um kernel é limitado por dois tetos principais: a largura de banda de memória e o pico computacional do hardware.

A intensidade aritmética pode ser escrita como:

$$I = \frac{FLOPs}{Bytes}. \quad (2.3)$$

Quando I é baixo, o kernel tende a ser *memory-bound*: seu desempenho é limitado pela taxa com que dados são fornecidos ao processador. Quando I é alto, o kernel tende a ser *compute-bound*: a limitação passa a ser a capacidade das unidades aritméticas. A

fronteira entre esses regimes ocorre no ponto em que o produto entre intensidade aritmética e largura de banda de memória atinge o pico computacional do dispositivo.

No contexto da atenção em ViTs, o Modelo Roofline permite avaliar se uma otimização melhora o desempenho por aumentar o uso das unidades aritméticas ou por reduzir o tráfego de memória. Um kernel fundido pode deslocar a operação no gráfico Roofline ao evitar a materialização da matriz $N \times N$, aumentando a intensidade aritmética efetiva e reduzindo a dependência da HBM. Por esse motivo, o modelo é adotado neste trabalho como ferramenta complementar às métricas de latência e throughput.

2.5 OpenAI Triton

Historicamente, a escrita de kernels otimizados exigia o uso de CUDA C++, uma linguagem de alta complexidade que demanda gerenciamento manual de threads e sincronização de blocos. O OpenAI Triton surge como uma alternativa de abstração que permite a escrita de kernels altamente performáticos utilizando uma sintaxe baseada em Python.

De acordo com Tillet, Kung e Cox (2019), o Triton introduz um paradigma baseado em blocos (tiles), onde o compilador é responsável por otimizar automaticamente o agendamento de tarefas e o uso dos registradores, permitindo que o desenvolvedor foque na lógica de movimentação de dados na hierarquia de memória.

Na prática, um kernel Triton é escrito como um programa parametrizado por blocos de dados. Cada instância do programa identifica sua região de atuação por meio de índices como `program_id`, carrega blocos com `tl.load`, executa operações vetoriais ou matriciais com primitivas como `tl.dot` e grava o resultado com `tl.store`. Máscaras booleanas são utilizadas para tratar bordas quando as dimensões do tensor não são múltiplas exatas do tamanho do bloco.

Parâmetros como `BLOCK_M`, `BLOCK_N`, `num_warps` e `num_stages` controlam a granularidade do trabalho e influenciam ocupação, pressão de registradores e paralelismo. Assim, o Triton oferece uma abstração intermediária: mais próxima da hierarquia de memória do que APIs de alto nível, mas menos verbosa que CUDA C++. Essa característica o torna adequado para o objetivo do trabalho, que é experimentar uma organização de dados inspirada em FlashAttention sem deslocar o foco para detalhes excessivos de sincronização manual de threads.

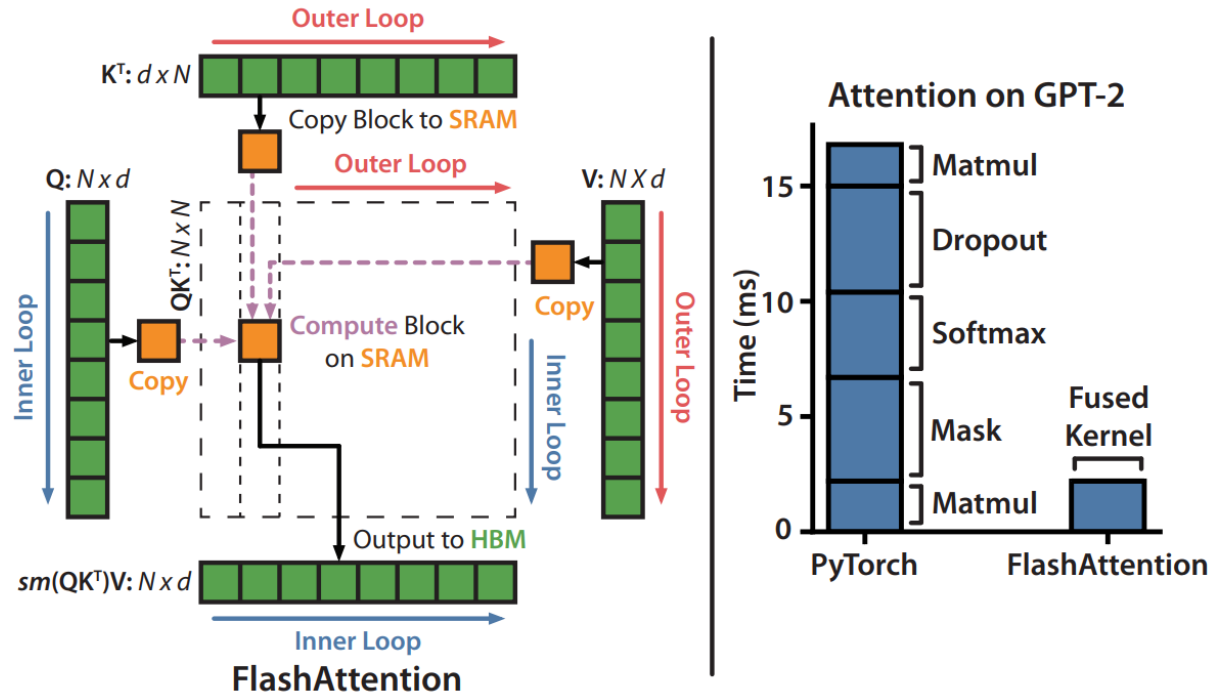
2.6 Fusão de Operadores e Flash Attention

A técnica de fusão de operadores (Operator Fusion) consiste em agrupar múltiplas operações matemáticas em um único kernel executável. No contexto de modelos de visão,

isso significa realizar o cálculo do Softmax e a multiplicação por V enquanto os dados de Q ainda residem nos registradores da GPU.

Como referência visual do método, a Figura 4 ilustra a hierarquia de memória, o particionamento em blocos e o ganho de desempenho obtido pelo kernel fundido do FlashAttention.

Figura 4 – Esquema do FlashAttention e comparação de tempo de execução



Fonte: Adaptado de Dao *et al.* (2022).

Esta técnica é levada ao estado da arte pelo algoritmo Flash Attention. Como sintetizado por Dao *et al.* (2022), a fusão de operadores permite que o modelo evite a materialização da matriz de atenção $N \times N$ na memória principal, resultando em acelerações significativas em relação às implementações tradicionais, reduzindo simultaneamente o consumo de memória.

3 METODOLOGIA

3.1 Ambiente Experimental

O ambiente experimental deste trabalho foi definido com o objetivo de isolar o impacto da implementação do kernel sobre a latência da operação de atenção. Os experimentos devem ser executados em uma GPU NVIDIA compatível com CUDA, utilizando PyTorch como baseline de alto nível e OpenAI Triton como ferramenta de desenvolvimento do kernel customizado. Para garantir reprodutibilidade, cada execução deve registrar o modelo da GPU, quantidade de SMs, versão do driver, versão do CUDA, versão do PyTorch, versão do Triton, tipo numérico utilizado e parâmetros de forma dos tensores.

Os tensores de entrada são organizados no formato $[B, H, N, D]$, em que B representa o tamanho do lote, H o número de cabeças de atenção, N o número de tokens e D a dimensão de cada cabeça. Esse formato é compatível com implementações usuais de Multi-Head Attention em modelos ViT. Para representar configurações típicas de visão computacional, consideram-se sequências como $N = 197$, correspondente a uma imagem de 224×224 particionada em patches de 16×16 acrescida do token de classe, e sequências maiores para avaliar o comportamento quadrático da atenção.

As medições de tempo devem ser feitas com eventos CUDA, após uma etapa de aquecimento destinada a amortizar efeitos de compilação JIT, inicialização de contexto e variações transientes de frequência. Quando disponível, o perfil de baixo nível deve ser coletado com ferramentas como NVIDIA Nsight Compute, registrando métricas de tráfego em memória global, ocupação, utilização de SMs e throughput computacional.

3.2 Design do Kernel Customizado

O kernel customizado implementa a etapa forward de atenção densa não causal, sem dropout, voltada ao cenário de inferência. O cálculo corresponde à Equação 2.1, porém evita a materialização completa da matriz QK^T , de dimensão $N \times N$, na memória global. A operação é decomposta em blocos de linhas de Q e blocos de colunas de K e V , permitindo que cada programa Triton carregue apenas uma fração da sequência por vez.

A estratégia de tiling utiliza dois parâmetros principais: $BLOCK_M$, que define a quantidade de queries processadas por programa, e $BLOCK_N$, que define a quantidade de keys e values carregados a cada iteração. Para cada bloco de queries, o kernel percorre a sequência de keys em blocos sucessivos, acumulando o resultado parcial da atenção. Desse modo, os valores intermediários necessários ao softmax permanecem em registradores ou SRAM durante a execução do programa, reduzindo leituras e escritas na HBM.

O escopo inicial da implementação considera tensores contíguos com eixos de lote, cabeças, sequência e dimensão por cabeça, além de dimensões de cabeça até 128 e tipos numéricos de baixa precisão, como FP16 e BF16. Essa delimitação é compatível com o objetivo experimental do trabalho: comparar o impacto da fusão de operadores e da consciência da hierarquia de memória em um caso representativo de atenção em Vision Transformers.

3.3 Implementação do Softmax Online

A comparação detalhada entre uma implementação padrão de *softmax* em PyTorch e uma versão otimizada com OpenAI Triton é apresentada no Apêndice A. O apêndice inclui os trechos de código e uma análise objetiva dos impactos em movimentação de memória, paralelismo e ocupação do hardware.

No kernel de atenção, entretanto, o *softmax* não é computado como uma operação isolada. Em vez de armazenar a matriz de scores completa, utiliza-se a formulação de *softmax online* empregada pelo FlashAttention (Dao *et al.*, 2022). Para cada linha da matriz de atenção, o kernel mantém três acumuladores: o maior valor observado m_i , a soma normalizada l_i e o acumulador vetorial de saída O_i .

Ao processar um novo bloco de scores S_{ij} , o máximo da linha é atualizado de forma incremental:

$$m_i^{novo} = \max \left(m_i^{antigo}, \max_j S_{ij} \right). \quad (3.1)$$

Em seguida, a soma exponencial anterior é reescalada para a nova referência numérica:

$$l_i^{novo} = e^{m_i^{antigo} - m_i^{novo}} l_i^{antigo} + \sum_j e^{S_{ij} - m_i^{novo}}. \quad (3.2)$$

O acumulador da saída passa pelo mesmo reescalonamento:

$$O_i^{novo} = e^{m_i^{antigo} - m_i^{novo}} O_i^{antigo} + \sum_j e^{S_{ij} - m_i^{novo}} V_j. \quad (3.3)$$

Ao final da varredura sobre todos os blocos de keys e values, a saída final é obtida por O_i/l_i . Essa formulação preserva a estabilidade numérica do *softmax* tradicional, pois continua subtraindo o maior valor da linha, mas permite que a normalização seja realizada em fluxo, sem escrita da matriz intermediária de atenção na memória global.

3.4 Protocolo de Avaliação

O protocolo de avaliação compara três implementações: uma versão explícita em PyTorch, que materializa as etapas QK^T , *softmax* e multiplicação por V ; a função otimizada `scaled_dot_product_attention` do PyTorch; e o kernel customizado em Triton. A primeira serve como referência de corretude e como baseline de alto nível mais transparente. A segunda representa uma implementação de biblioteca otimizada. A terceira corresponde à contribuição experimental deste trabalho.

Para cada configuração de B , H , N , D e tipo numérico, são gerados tensores aleatórios em GPU e executadas rodadas de aquecimento antes das medições. A latência média é calculada a partir de múltiplas iterações sincronizadas por eventos CUDA. A corretude é verificada pelo erro absoluto máximo entre a saída do kernel avaliado e a implementação explícita em PyTorch.

Além da latência, calcula-se o throughput aproximado em TFLOP/s. Considerando que a atenção densa realiza uma multiplicação QK^T e uma multiplicação PV , ambas com custo aproximado de $2BHN^2D$ operações de ponto flutuante, o custo total do forward é estimado como:

$$FLOPs \approx 4BHN^2D. \quad (3.4)$$

Essa estimativa permite comparar o desempenho obtido com o limite teórico da GPU. A interpretação final deve combinar os resultados de latência, throughput, tráfego de memória global e Modelo Roofline, distinguindo casos *memory-bound* de casos *compute-bound*.

4 RESULTADOS E DISCUSSÕES

4.1 Análise Comparativa de Latência

4.2 Profiling de Baixo Nível

4.3 Modelo Roofline

5 CONCLUSÃO

Apresentar as conclusões correspondentes aos objetivos ou hipóteses propostos para o desenvolvimento do trabalho, podendo incluir sugestões para novas pesquisas.

REFERÊNCIAS

DAO, T. *et al.* **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. Disponível em: <https://arxiv.org/abs/2205.14135>.

DOSOVITSKIY, A. *et al.* **An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**. 2021. Disponível em: <https://arxiv.org/abs/2010.11929>.

TILLET, P.; KUNG, H. T.; COX, D. Triton: an intermediate language and compiler for tiled neural network computations. *In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. New York, NY, USA: Association for Computing Machinery, 2019. (MAPL 2019), p. 10–19. ISBN 9781450367196. Disponível em: <https://doi.org/10.1145/3315508.3329973>.

VASWANI, A. *et al.* **Attention Is All You Need**. 2023. Disponível em: <https://arxiv.org/abs/1706.03762>.

WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: an insightful visual performance model for multicore architectures. **Communications of the ACM**, Association for Computing Machinery, New York, NY, USA, v. 52, n. 4, p. 65–76, 04 2009. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/1498765.1498785>.

WULF, W.; MCKEE, S. Hitting the memory wall: Implications of the obvious. **Computer Architecture News**, v. 23, 01 1996.

APÊNDICES

APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E SOFTMAX COM TRITON

Este apêndice apresenta uma comparação entre duas abordagens para cálculo de *softmax* em linhas de uma matriz: uma implementação padrão em PyTorch e uma implementação otimizada com OpenAI Triton.

A.1 Implementação padrão em PyTorch

```
import torch

import triton
import triton.language as tl
from triton.runtime import driver

DEVICE = triton.runtime.driver.active.get_active_torch_device()

def is_hip():
    return (
        triton.runtime.driver.active.get_current_target().backend == "hip"
    )

def is_cdna():
    return (
        is_hip()
        and triton.runtime.driver.active.get_current_target().arch in (
            'gfx940', 'gfx941', 'gfx942', 'gfx90a', 'gfx908'
        )
    )

def naive_softmax(x):
    """Compute row-wise softmax of X using native pytorch

    We subtract the maximum element in order to avoid overflows. Softmax is
    invariant to this shift.
```

```
"""
# read MN elements ; write M elements
x_max = x.max(dim=1)[0]
# read MN + M elements ; write MN elements
z = x - x_max[:, None]
# read MN elements ; write MN elements
numerator = torch.exp(z)
# read MN elements ; write M elements
denominator = numerator.sum(dim=1)
# read MN + M elements ; write MN elements
ret = numerator / denominator[:, None]
# in total: read 5MN + 2M elements ; wrote 3MN + 2M elements
return ret
```

A.2 Implementação otimizada com Triton

```
properties = driver.active.utils.get_device_properties(DEVICE.index)
NUM_SM = properties["multiprocessor_count"]
NUM_REGS = properties["max_num_regs"]
SIZE_SMEM = properties["max_shared_mem"]
WARP_SIZE = properties["warpSize"]
target = triton.runtime.driver.active.get_current_target()
kernels = {}

def softmax(x):
    n_rows, n_cols = x.shape

    # The block size of each loop iteration is the smallest power of two
    # greater than the number of columns in 'x'
    BLOCK_SIZE = triton.next_power_of_2(n_cols)

    # Another trick we can use is to ask the compiler to use more threads
    # per row by increasing the number of warps ('num_warps')
    # over which each row is distributed.
    num_warps = 8

    # Number of software pipelining stages.
    num_stages = 4 if SIZE_SMEM > 200000 else 2
```

```

# Allocate output
y = torch.empty_like(x)

# pre-compile kernel to get register usage
# and compute thread occupancy.
kernel = softmax_kernel.warmup(
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE=BLOCK_SIZE, num_stages=num_stages,
    num_warps=num_warps, grid=(1, )
)
kernel._init_handles()
n_regs = kernel.n_regs
size_smem = kernel.metadata.shared

if is_hip():
    # NUM_REGS represents the number of regular purpose registers.
    # On CDNA architectures this is half of all registers available.
    NUM_GPRS = NUM_REGS
    if is_cdna():
        NUM_GPRS = NUM_REGS * 2

    # MAX_NUM_THREADS represents maximum resident threads per SM.
    MAX_NUM_THREADS = properties["max_threads_per_sm"]
    max_num_waves = MAX_NUM_THREADS // WARP_SIZE
    occupancy = min(NUM_GPRS // WARP_SIZE // n_regs, max_num_waves)
    occupancy = occupancy // num_warps
else:
    occupancy = NUM_REGS // (n_regs * WARP_SIZE * num_warps)

occupancy = min(occupancy, SIZE_SMEM // size_smem)
num_programs = NUM_SM * occupancy
num_programs = min(num_programs, n_rows)

# Create a number of persistent programs.
kernel[(num_programs, 1, 1)](
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE, num_stages
)

```

```
return y
```

A.3 Comparação técnica

- **Movimentação de memória:** a versão padrão materializa tensores intermediários para cada etapa; a versão Triton busca reduzir leituras e escritas na HBM por meio de execução em blocos e reutilização de dados em SRAM.
- **Paralelismo:** na implementação Triton, parâmetros como `BLOCK_SIZE`, `num_warps` e `num_stages` são ajustados para aumentar ocupação e throughput.
- **Consciência de hardware:** o código Triton utiliza propriedades do dispositivo (SMs, registradores e memória compartilhada) para definir o número de programas persistentes, aproximando a implementação do limite arquitetural da GPU.
- **Objetivo prático:** enquanto a versão padrão prioriza simplicidade e legibilidade, a versão Triton prioriza eficiência computacional e redução da latência em inferência.

ANEXOS

ANEXO A – EXEMPLO DE ANEXO

Elemento opcional, que consiste em um texto ou documento não elaborado pelo autor, que serve de fundamentação, comprovação e ilustração, conforme a ABNT NBR 14724. (??).

O **ANEXO B** exemplifica como incluir um anexo em pdf.

ANEXO B – ACENTUAÇÃO (MODO TEXTO - $\text{\LaTeX}$)

Figura 5 – Acentuação (modo texto - $\text{\LaTeX}$)

$\text{\textbackslash'a}$ - á
 $\text{\textbackslash'a}$ - à
 $\text{\textbackslash~a}$ - ã
 $\text{\textbackslash^a}$ - â
 $\text{\textbackslash'e}$ - é
 $\text{\textbackslash^e}$ - ê
 $\text{\textbackslash'\{i\}}$ - í
 $\text{\textbackslash'I}$ - Í
 $\text{\textbackslash'o}$ - ó
 $\text{\textbackslash~o}$ - õ
 $\text{\textbackslash^o}$ - ô
 $\text{\textbackslash'u}$ - ú
 $\text{\textbackslash"u}$ - ü
 $\text{\textbackslashc{c}}$ - ç
 $\text{\textbackslashc{C}}$ - Ç

Fonte: Comandos [...] (??)